CYBERSECURITY INTERNSHIP — Complete Report

Name: Hammad Rasheed

ID: DHC-5590

Internship Type: Online Cybersecurity Internship

Week 1

Security Assessment

Step 1

Setup the Application

Prerequisites

- Node.js installed
- npm installed

Installation & Startup Commands

```
# Clone the repository
git clone https://github.com/juice-shop/juice-shop.git --
depth 1

# Enter the directory
cd juice-shop

# Install dependencies (first time only)
npm install

# Start the application
npm start
```

Open the application in your browser:

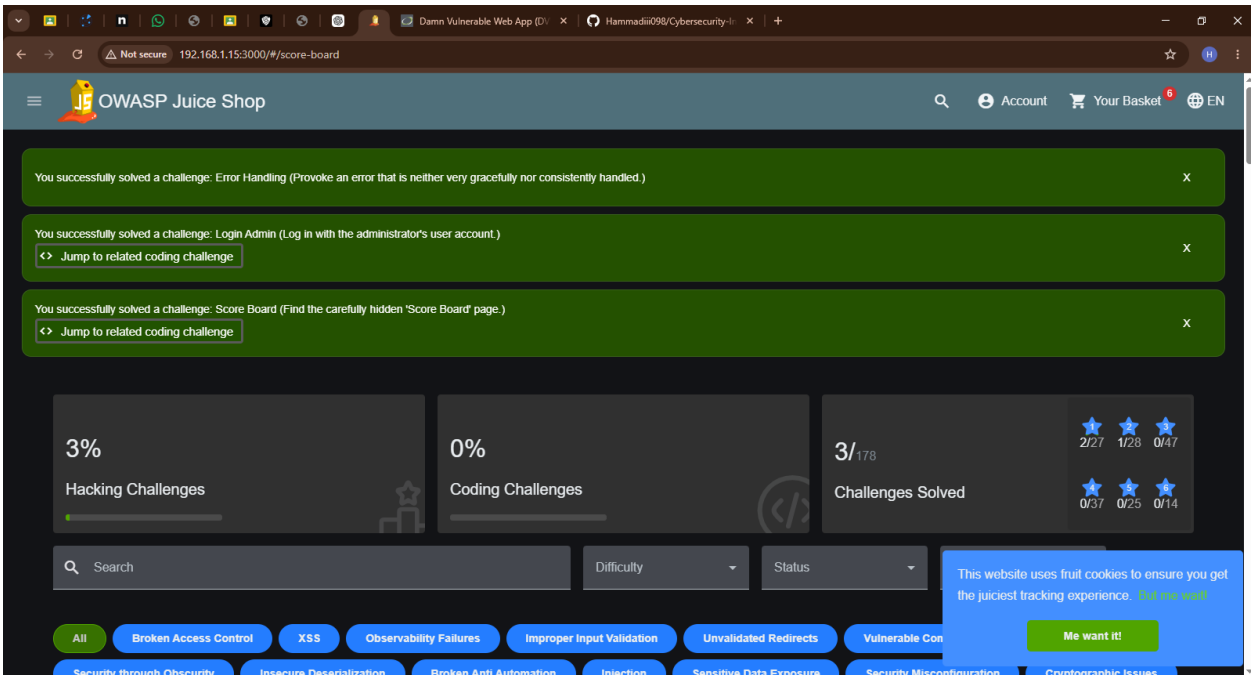
http://localhost:3000

Explore the following sections:

- Signup
- Login
- Search
- Basket
- Profile

Score Board URL:

http://localhost:3000/#/score-board



Step 2: Vulnerability Assessment

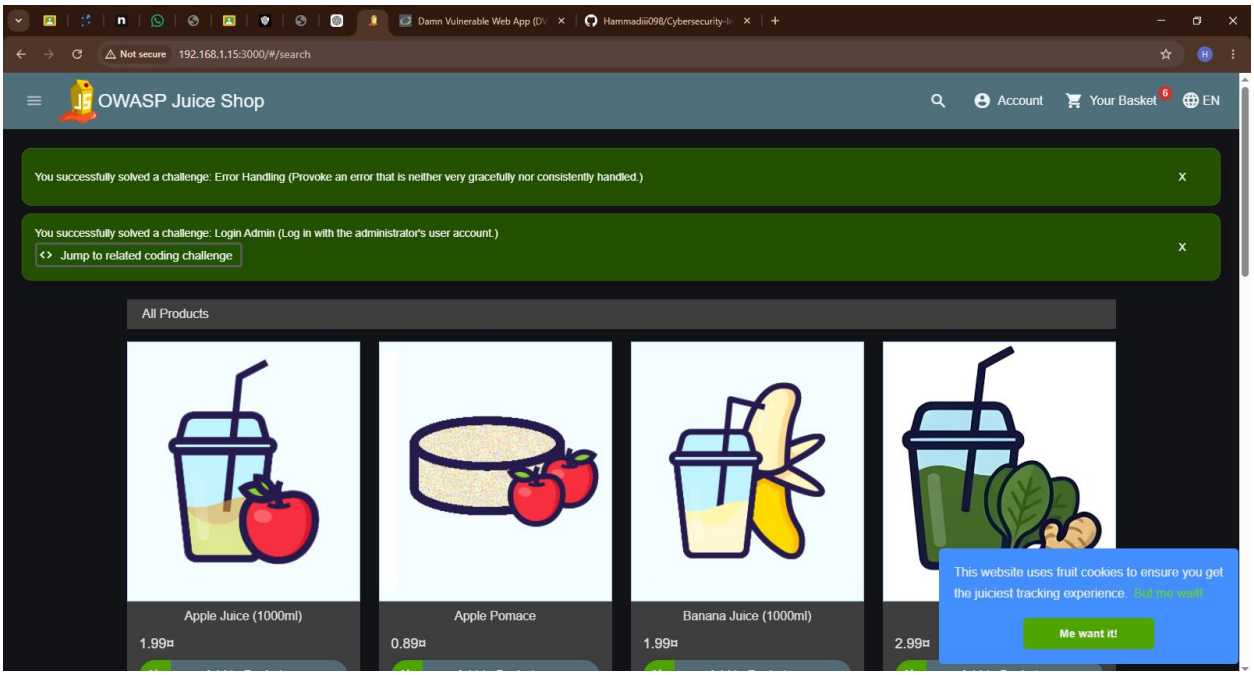
A. SQL Injection — Login as Admin

Steps

1. Open the Login page
2. Enter the following payload in the Email field:

```
' OR 1=1;--
```

3. Enter any password
4. Click **Log in**



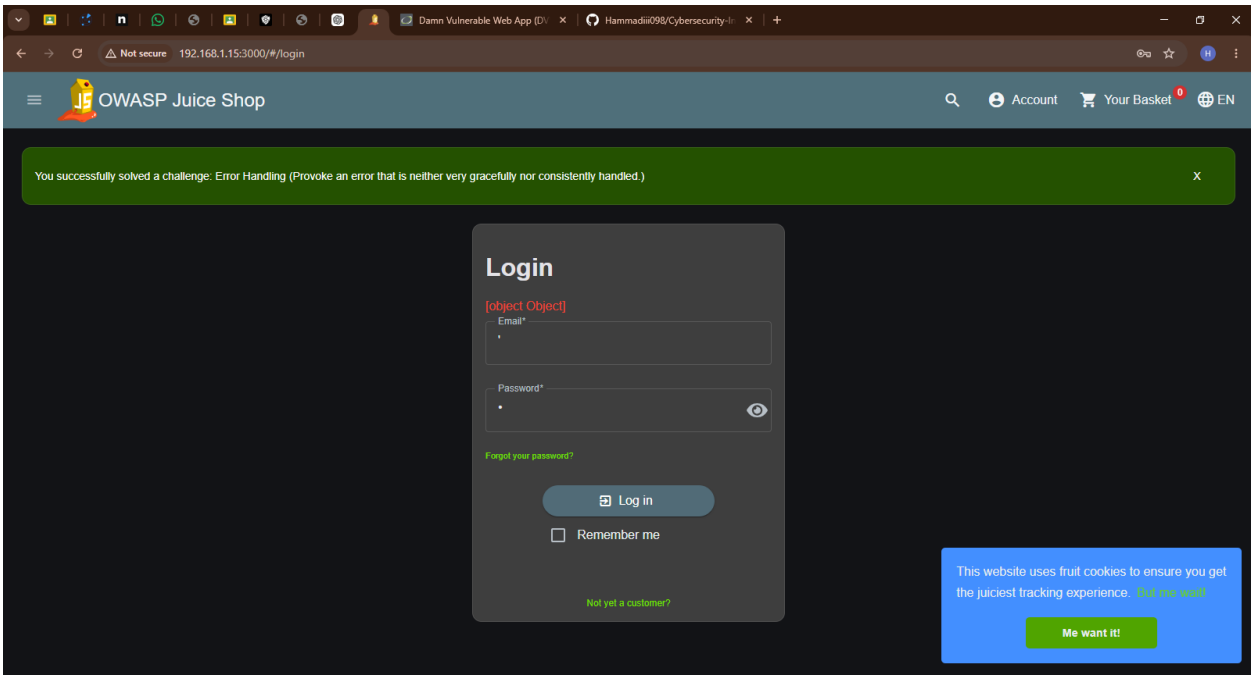
What Happens

The SQL query becomes:

```
SELECT * FROM Users
WHERE email = '' OR 1=1;--'
AND password = '...'
AND deletedAt IS NULL
```

Explanation

- `' OR 1=1` always evaluates to TRUE
- `--` comments out the remaining query
- Authentication is bypassed



Alternative Payload

admin@juice-sh.op' --

Admin Panel

After login, open:

http://localhost:3000/#/administration

B. Cross-Site Scripting (XSS) — DOM XSS

Search Bar XSS

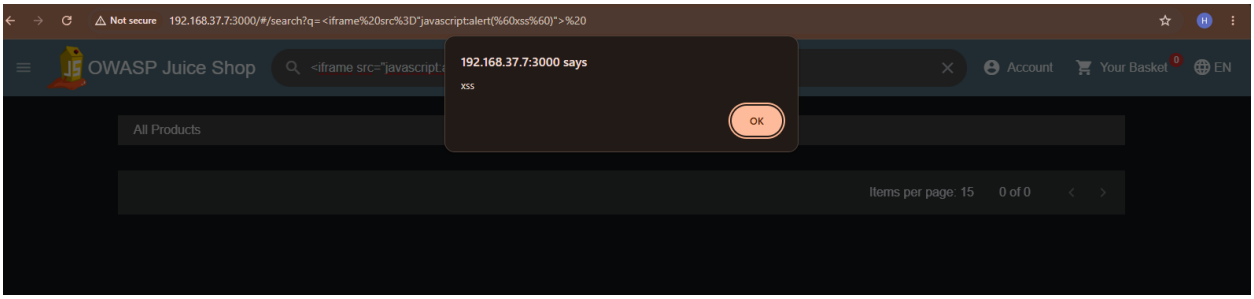
1. Click the Search icon
2. Enter:

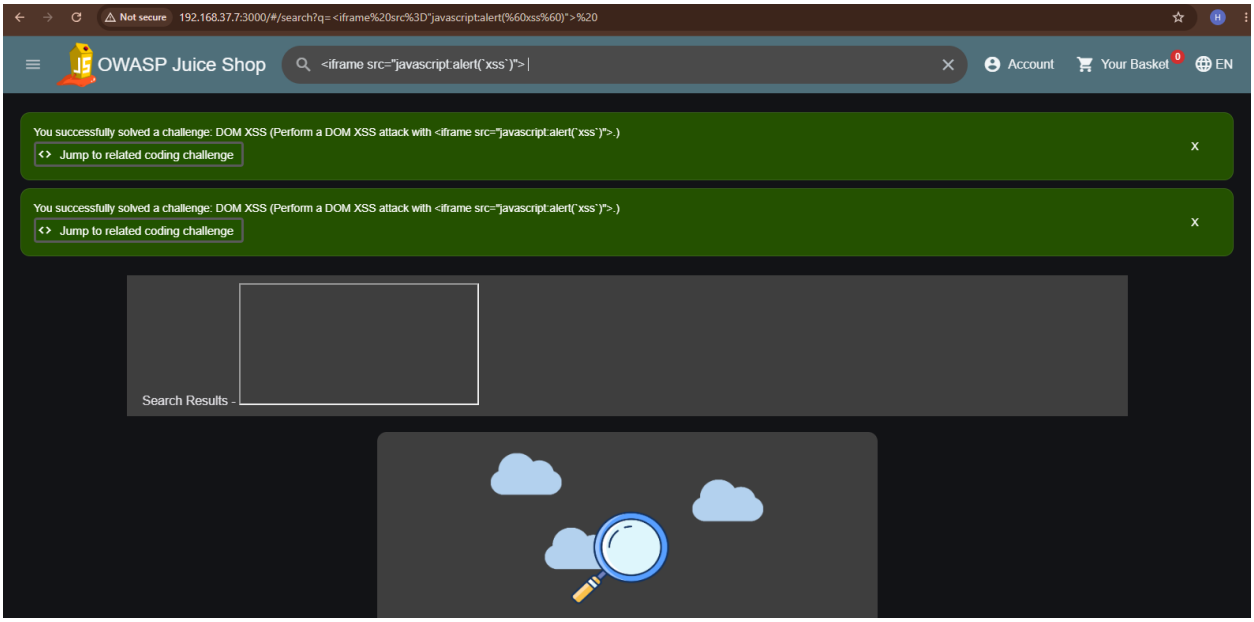
```
<iframe src="javascript:alert(`xss`)">
```

3. Press Enter

Result

JavaScript executes in the browser due to unsanitized DOM rendering.





Reflected XSS — Track Order Page

Inject the following into the tracking field:

```
<script>alert('XSS');</script>
```

C. Browser Developer Tools Inspection

Open DevTools

- Right-click → Inspect
- Or press `F12`

Areas to Inspect

Sources Tab

Search inside `juice-shop.min.js` for:

- admin
- password
- token
- API endpoints

Network Tab

Inspect:

- API responses
- JWT tokens
- User data

Application Tab

Check:

- Local Storage
- Session Storage

D. Automated Scanning with OWASP ZAP

Install ZAP

Download from:

https://www.zaproxy.org/download/

Quick Scan

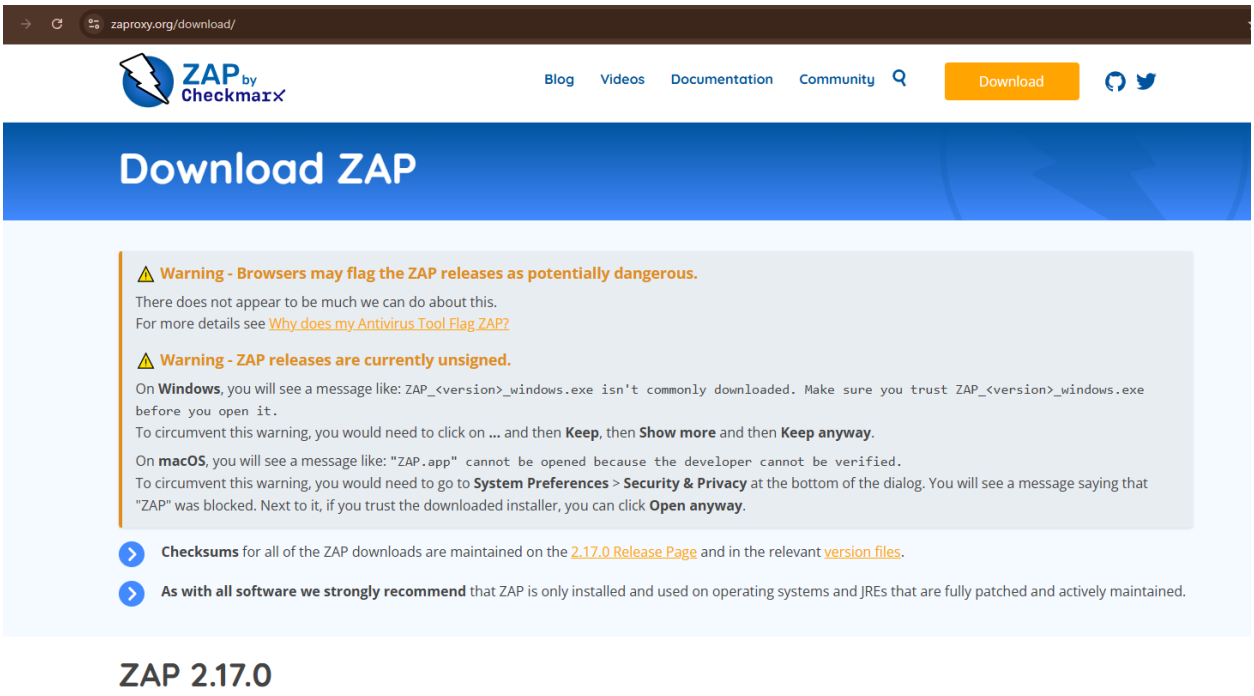
```
Tools → Quick URL Scan
```

Target:

http://localhost:3000

Full Scan

```
File → New Session
Sites Tree → Right-click → Attack → Active Scan
```



Review Findings

Check the **Alerts** tab:

- High
- Medium
- Low risk vulnerabilities

Export Report

`Report → Generate Report → HTML`

E. Weak Password Storage

Juice Shop intentionally uses weak MD5 hashing.

Example Endpoints

```
/api/Users
/rest/product/search
```

Crack MD5 Hashes

Use:

https://crackstation.net

Known Credentials

Email	Password
admin@juice-sh.op	admin123
jim@juice-sh.op	ncc-1701
bender@juice-sh.op	OhG0dPlease1nsertLol

Step 3: Document Findings

#	Vulnerability	Location	Severity	Description
1	SQL Injection	/rest/user/login	Critical	Authentication bypass using SQL payload
2	DOM XSS	Search Bar	High	Unsanitized rendering allows script execution
3	Reflected XSS	Track Order Page	High	User input reflected without sanitization
4	Weak Password Hashing	User Database	High	MD5 hashes are easily crackable
5	Exposed Admin Section	/administration	Medium	Weak access control
6	JWT Weak Secret	/rest/user/login	Medium	Predictable JWT secret
7	CORS Misconfiguration	API Endpoints	Medium	Overly permissive CORS
8	Missing Security Headers	All Pages	Low	Missing CSP and clickjacking protections

Areas of Improvement

- Parameterized queries
- Input validation
- Output encoding
- bcrypt/argon2 password hashing
- Strong JWT secrets
- RBAC implementation
- Helmet.js security headers

Week 2

Implementing Security Measures

1. Input Validation & Sanitization

```
const validator = require('validator');

app.post('/api/Users', async (req, res) => {
  const { email, password } = req.body;

  if (!email || !validator.isEmail(email)) {
    return res.status(400).send('Invalid email format');
  }

  const sanitizedEmail = validator.escape(email);
  const sanitizedPassword = validator.escape(password);

  // Continue safely...
});
```

2. Password Hashing with bcrypt

Install

```
npm install bcrypt
```

Secure Registration

```
const bcrypt = require('bcrypt');
const saltRounds = 10;

const hashedPassword = await bcrypt.hash(password,
saltRounds);
```

Secure Login

```
const match = await bcrypt.compare(password, user.password);
```

3. JWT Token-Based Authentication

Install

```
npm install jsonwebtoken
```

Generate JWT

```
const jwt = require('jsonwebtoken');

const token = jwt.sign(
  { id: user.id, email: user.email, role: user.role },
  JWT_SECRET,
  { expiresIn: '1h' }
);
```

Verify JWT

```
jwt.verify(token, JWT_SECRET, (err, decoded) => {
  if (err) return res.status(403).send('Invalid token');
});
```

4. Secure HTTP Headers with Helmet

Install

```
npm install helmet
```

Usage

```
const helmet = require('helmet');

app.use(helmet());
```

Important Headers

- X-Frame-Options
- CSP
- HSTS
- X-Content-Type-Options

5. SQL Injection Prevention

Vulnerable Code

```
const query = `SELECT * FROM Users
WHERE email='${email}' AND password='${password}'`;
```

Secure Version

```
const query = 'SELECT * FROM Users WHERE email = ? AND
password = ?';

db.query(query, [email, hashedPassword]);
```

Better Approach

```
const user = await User.findOne({
  where: { email }
});
```

Week 3

Advanced Security & Final Reporting

1. Basic Penetration Testing with Nmap

Scan Open Ports

```
nmap -p- 127.0.0.1
```

Service Detection

```
nmap -sV -p 3000 127.0.0.1
```

Vulnerability Scan

```
nmap --script http-vuln* -p 3000 127.0.0.1
```

2. Cracking Weak Hashes

Hashcat

```
hashcat -m 0 -a 0 hashes.txt rockyou.txt
```

John the Ripper

```
john --format=raw-md5 --wordlist=rockyou.txt hashes.txt
```

3. JWT Manipulation

Brute Force JWT Secret

```
hashcat -m 16500 -a 0 jwt.txt rockyou.txt
```

4. Logging with Winston

Install

```
npm install winston
```

Example Logger

```
const winston = require('winston');

const logger = winston.createLogger({
  level: 'info',
  transports: [
```

```
    new winston.transports.Console(),
    new winston.transports.File({
      filename: 'security.log'
    })
  ]
});
```

Log Failed Login Attempts

```
logger.warn('Failed login attempt', {
  email,
  ip: req.ip
});
```

Final Security Checklist

#	Best Practice	Status	Notes
1	Input Validation	✅	Validate all inputs
2	Output Encoding	✅	Escape HTML entities
3	Parameterized Queries	✅	Avoid raw SQL
4	Password Hashing	✅	Use bcrypt
5	JWT Security	✅	Strong secrets + expiry
6	HTTPS/TLS	✅	Enforce HTTPS
7	Security Headers	✅	Use Helmet.js
8	Access Control	✅	RBAC implementation
9	CORS Configuration	✅	Restrict origins
10	Rate Limiting	✅	Protect login/API
11	Error Handling	✅	Generic messages
12	Logging & Monitoring	✅	Monitor suspicious activity

13	Session Management	✅	Secure cookies
14	Dependency Scanning	✅	Run npm audit
15	CSRF Protection	✅	Use CSRF tokens

Quick Reference — Key Challenges & Solutions

Challenge	Method	Payload / Technique
Score Board	URL Guess	`/#/score-board`
Login Admin	SQLi	`' OR 1=1;--`
DOM XSS	Search Bar	`<iframe src="javascript:alert('xss')">`
Admin Section	Navigation	`/#/administration`
Confidential Document	Path Traversal	`ftp/legal.md`
Error Handling	Invalid Input	`'` in login field
Exposed Credentials	OSINT	Inspect JS files
Zero Stars	Burp Interception	Rating = 0
Forged Feedback	IDOR	Modify UserId
View Basket	IDOR	Modify BasketId